

Testing Overview

The rewind system was tested again after expanding the project from a basic rewind feature into a more complete puzzle mechanic. In the earlier version, the rewind worked technically, but it was not connected enough to gameplay. The new version now includes more reasons for the player to use time reset, such as recovering the cube, rewinding platforms, opening doors, and solving laser barrier puzzles.

The main goal of testing was to check that the rewind system still works correctly while also checking if the new gameplay elements make sense. This includes player movement, cube pickup and throwing, physics objects, UI text, world-space UI hints, audio feedback, platform rewind, pressure plates, laser barriers, and edge cases like spamming inputs or losing the cube.

The system still uses frame-by-frame recording instead of interpolation because this gave more accurate results. Actors move back along their real previous path, which is important for the cube and for the final laser puzzle where the cube's rewind path is part of the solution.

Feedback Addressed

The feedback from the previous version said that the feature was framed too small and needed more elements that belong to it. To improve this, I added actual puzzle situations where time reset is needed. The player now has to use rewind to recover the cube, move platforms back into position, and reverse the cube's path to solve the final laser barrier puzzle.

The feedback also mentioned that there was not much audio and not enough consistent visualisation. I improved this by adding button sounds, door sounds, rewind sound, cube rewind trail effects, UI mode text, a rewind bar, platform status text, world-space instructions, and laser visuals that turn on and off.

The feedback also said that the testing was not deep enough. Because of that, this updated test plan now includes both technical tests and gameplay tests. It also includes user testing, where players are asked if they understand why rewind is needed and whether the UI and world hints are clear.

Test Coverage Justification

The tests were chosen to cover the most important parts of the improved project. The original tests focused mostly on whether the rewind system worked correctly. This was still important, so those tests were kept and updated. However, the new version also needed tests for the actual gameplay built around rewind.

The new tests cover cube interaction, doors, buttons, platform rewind, laser barriers, pressure plates, world UI, sounds, and puzzle clarity. These are important because they show whether the rewind system is now useful as a game mechanic and not just a technical feature.

Some tests, like multiplayer behaviour, were still left out because the project is single player. Very large stress tests with many actors were also not the main focus because the level is designed as a small puzzle prototype. However, performance and memory use are still mentioned in the evaluation because frame-based recording can become expensive in bigger projects.

Test Table

T1 – Player Rewind

Moved the player around and then held rewind in Player mode.

Expected: player goes back along the path it came from.

Actual: player followed the previous movement path backwards.

Metrics: around 600 stored frames with 0.02 record interval.

Result: Pass

T2 – Cube Rewind

Threw the cube, switched to Cube mode, and held rewind.

Expected: cube moves backwards along the path it travelled.

Actual: cube returned along its throw path correctly.

Metrics: cube used stored frame data from RewindComponentCPP.

Result: Pass

T3 – Multi-Actor Rewind

Moved the player and cube, then used All mode.

Expected: all rewindable actors rewind together.
Actual: player and cube rewound at the same time.
Metrics: all actors with the Rewind tag were included.
Result: Pass

T4 – Physics Restore

Threw the cube, rewound it, then released rewind.
Expected: cube should not freeze, slide forever, or keep unwanted velocity.
Actual: at first, the cube sometimes kept a bit of velocity after rewind ended. This was fixed by resetting linear and angular velocity when the cube stopped rewinding.
Metrics: linear and angular velocity reset after rewind ended.
Result: Initial Partial Pass, Retest Pass

T5 – Empty Rewind

Pressed rewind with very little stored frame data.
Expected: system does not crash or break.
Actual: rewind stopped quickly without crashing, but feedback could still be clearer because the player does not get a strong message saying there is no rewind history.
Metrics: fewer than 5 stored frames.
Result: Partial Pass

T6 – Rapid Input

Pressed rewind and switched modes quickly.
Expected: system remains stable.
Actual: system stayed stable, with only minor possible UI flicker during very fast input. It did not break the rewind state.
Metrics: around 10–15 rapid input changes tested.
Result: Partial Pass

T7 – Cube Pickup and Hold

Walked into the cube and waited after collecting it.
Expected: cube is stored and does not fall, spin, or collide while held.
Actual: cube was stored correctly with physics, gravity, and collision disabled.
Metrics: cube inventory set to 1, cube hidden while held.
Result: Pass

T8 – Cube Throw

Picked up the cube and pressed throw.
Expected: cube appears from the player/hold point and travels forward.
Actual: cube was thrown in the camera direction. It took some adjustment to stop it

spawning too far from the player.

Metrics: physics, gravity, collision, and velocity restored on throw.

Result: Pass

T9 – Cube Pickup Cooldown

Threw the cube close to the player.

Expected: cube should not instantly collect itself again.

Actual: at first, the cube sometimes got picked up immediately after being thrown because the pickup range was still active. This was fixed by adding a short pickup cooldown after throw.

Metrics: collection disabled briefly after throw.

Result: Initial Fail, Retest Pass

T10 – Pickup Disabled During Rewind

Rewound the cube near the player.

Expected: cube should not auto-pickup while rewinding.

Actual: at first, the cube could freeze near the player because it was being collected during rewind. This was fixed by checking if the cube was rewinding before allowing pickup.

Metrics: pickup checks rewind state before collecting.

Result: Initial Fail, Retest Pass

T11 – Rewind History Reset After Throw

Picked up the cube, threw it again, and then rewound.

Expected: cube rewinds back to the latest throw point instead of old stored positions.

Actual: at first, the cube sometimes jumped back to older recorded frames. This was fixed by clearing old stored frames when the cube is thrown again.

Metrics: rewind data reset when cube is thrown.

Result: Initial Partial Pass, Retest Pass

T12 – Rewind Mode Filtering

Tested All, Player, Cube, and Platforms mode.

Expected: only actors belonging to the selected mode rewind.

Actual: mode filtering worked correctly for all four modes.

Metrics: ActorMatchesRewindMode used actor tags.

Result: Pass

T13 – Platform Mode Unlock

Opened the first door and reached the platform section.

Expected: Platforms mode becomes available after this point.

Actual: Platforms mode was added to the mode cycle after the door opened.

Metrics: mode cycle increased from 3 states to 4 states.

Result: Pass

T14 – Button and Sliding Door

Threw the cube onto the button.

Expected: button activates and the linked double door opens smoothly.

Actual: button activated and the double door opened using a Timeline. Earlier, the door movement was too instant, so the Timeline made it feel more polished.

Metrics: button checked cube tag, door used Timeline/Lerp movement.

Result: Pass

T15 – Button and Door Audio

Activated the button and opened the door.

Expected: button and door sounds play at the correct moments.

Actual: button sound played on activation and door sound played during opening. This was added after feedback that the feature needed more audio.

Metrics: SoundBase variables triggered once per interaction.

Result: Pass

T16 – Platform Drop and Rewind

Opened the door, waited for platforms to drop, then switched to Platforms mode and held rewind.

Expected: platforms move back up and create a path.

Actual: platforms rewound back into place and the player could cross. At first, the rewind felt too slow if the player waited, so the platform rewind settings were adjusted.

Metrics: platform actors used RewindComponentCPP and Platform tag.

Result: Initial Partial Pass, Retest Pass

T17 – World-Space Platform Hint

Reached the platform section.

Expected: 3D UI tells the player to switch to Platforms mode.

Actual: world-space text appeared and explained the next action. This was added because testers did not immediately notice that Platforms mode had unlocked.

Metrics: Widget Component placed in the level.

Result: Pass

T18 – Laser Barrier Blocks Player

Walked into an active laser barrier.

Expected: player cannot pass through.

Actual: laser collision blocked the player.

Metrics: LaserBlocker collision set to block player.

Result: Pass

T19 – Pressure Plate Laser Toggle

Moved the cube onto and off a pressure plate.

Expected: linked laser disables while cube is on the plate and enables when cube leaves.

Actual: laser visual and collision toggled correctly. It took some adjustment to make sure all the laser visual pieces disabled together.

Metrics: plate called DisableLaser and EnableLaser on TargetLaser.

Result: Pass

T20 – Cube Rewind Laser Puzzle

Used the cube to disable the first laser, passed it, then rewound the cube onto the second plate.

Expected: cube's rewind path activates the second pressure plate.

Actual: the puzzle worked once the plate positions were adjusted to match the cube's actual rewind path. The cube path had to be tested a few times so the solution felt fair.

Metrics: final puzzle used cube history as part of the solution.

Result: Initial Partial Pass, Retest Pass

T21 – Full Level Flow

Played the level from start to finish.

Expected: player can complete door, platform, and laser puzzles using rewind.

Actual: full level was completed using cube throw, mode switching, platform rewind, and cube path rewind. A few small fixes were needed during testing, mainly around cube pickup and platform clarity.

Metrics: all major mechanics used at least once.

Result: Pass

T22 – Rewind Bar

Recorded movement and rewound in different modes.

Expected: bar shows stored rewind history for the selected mode.

Actual: at first, the bar only worked properly for Player and All because it was reading the player rewind component. This was changed so the bar reads the current selected rewind mode.

Metrics: percent based on $\text{StoredFrames} / \text{MaxStoredFrames}$.

Result: Initial Partial Pass, Retest Pass

T23 – Visual Feedback

Rewound the cube/player and toggled lasers.

Expected: rewind trail and laser visuals clearly show state changes.

Actual: Niagara rewind trail appeared during rewind and laser visuals hid/showed correctly. Some laser pieces had to be grouped/disabled together so the barrier looked fully off.

Metrics: trail and laser visibility toggled based on gameplay state.

Result: Pass

T24 – Missing Reference Safety

Temporarily removed a door or laser reference and triggered the interaction.

Expected: the game should not crash.

Actual: IsValid checks stopped broken references from causing runtime failure.

Metrics: blueprint references checked before function calls.

Result: Pass

T25 – User Clarity Test

Asked testers to play without explaining the full solution.

Expected: players understand the purpose of rewind with UI/world hints.

Actual: testers understood the first door puzzle quickly, but two testers needed extra hints for Platforms mode and the final laser puzzle. After this, world-space text and clearer instructions were added.

Metrics: 3 testers, 2/3 needed extra hints before changes.

Result: Partial Pass, improved after changes

User Testing

Three testers were asked to play the level and try to finish it without being told the full solution immediately. They were asked to say out loud if something was confusing or unclear.

Tester 1 completed the button and door section but got confused at the platform section. They did not immediately notice that Platforms mode had been unlocked. Based on this, world-space UI text was added to tell the player to press Q and switch to Platforms mode.

Tester 2 completed the platform section but was confused in the final laser room. They did not understand at first that the cube's rewind path was part of the solution. They tried to look for another button instead of rewinding the cube back onto the second pressure plate. Based on this, a final room hint was added to explain that the cube remembers its path.

Tester 3 completed the level after the extra hints were added, but said that the button and door felt too quiet and needed stronger feedback. Based on this, button and door sounds were added, and the laser visuals were made clearer.

The questions asked after testing were:

Did you understand why rewind was needed?

Did the rewind modes make sense?

Was it clear when to use Cube mode?

Was it clear when to use Platforms mode?

Did the UI help?

Did the 3D text help?

Did the audio make interactions clearer?

Was the final laser puzzle understandable?

Did anything feel broken or unfair?

Evaluation

Overall, the updated version works better because the rewind mechanic is now used as part of the level design. The earlier version mainly proved that rewind worked technically, but the player did not have many reasons to use it. The improved version gives the player several different reasons to use rewind: recovering the cube, restoring platforms, and using the cube's previous path to solve the laser barrier puzzle.

The frame-based rewind system still works well for this project because it gives accurate movement paths. This is especially important for the cube, because the final puzzle depends on the cube moving back along its real previous route. The downside is still performance and memory. Storing frame data for multiple actors uses more memory than interpolation. At 0.02 second intervals and hundreds of stored frames, the data can add up. For this project size it is acceptable, but a larger game would need optimisation.

The UI and feedback are much stronger now. The mode text helps the player understand what is currently being rewound, the rewind bar shows available rewind history, and world-space text explains puzzle-specific instructions. Audio feedback also makes the level feel clearer, especially with button sounds, door sounds, and the rewind sound. The rewind trail gives a more consistent visualisation of the time reset effect.

The testing was not perfect the first time. Some issues only became clear after actually playing the puzzle flow, especially the cube being collected too early, the platform mode

not being obvious, and the final laser puzzle needing better explanation. Those problems were fixed through cooldowns, world-space hints, and clearer feedback.

There are still improvements that could be made. The empty rewind state could have clearer feedback, such as a sound or UI message saying there is no rewind history available. The UI could also be polished more visually. Optimisation could also be improved by only recording actors when they are moving, compressing frame data, or using an adaptive recording rate.

Reflection

The biggest improvement was changing the rewind system from a small technical mechanic into a proper puzzle feature. Before, the system showed that time rewind worked, but it did not clearly show why the player needed it. The new version gives the player actual problems that rewind solves.

The cube became the main object that connects the level together. It can be picked up, thrown, rewound, used on buttons, used with pressure plates, and used to solve the laser room. This makes the mechanic feel more connected because the player has to think about where the cube was and where it will go when time is reversed.

The platform puzzle also helped because it showed that rewind can affect the environment, not just the player or cube. The final laser puzzle was useful because it made the cube's recorded path part of the solution. That made the rewind system feel more like a gameplay mechanic rather than only an effect.

The C++ RewindComponent supports the main recording and restoring system, while Blueprint handles the puzzle logic, UI, audio, and interactions. This worked well because the C++ part keeps the core mechanic reliable, while Blueprint made it faster to build and adjust level features.

If the project continued, I would focus on optimisation and polish. I would reduce memory usage by recording only moving actors, add better particles for time reset, improve the rewind bar animation, and add clearer feedback when the player tries to rewind with no stored frames.